



Systematic Smart Contract Auditing: Methods and Tools

Zach Kelling

Lux Industries

2025

Abstract

A smart contract audit is a systematic search for vulnerabilities in code that controls money. This paper presents a complete audit methodology: static analysis with Slither, fuzz testing with Foundry, pattern detection with Aderyn, symbolic execution with Halmos, and manual review. We describe the methodology applied to Lux Network’s 50 contract domains (AMM, bridge, vault, staking, governance, oracle, token, NFT, lending, and 41 more), comprising 832 tests and 122 findings across five severity levels. For each vulnerability class—reentrancy, flash loan attacks, oracle manipulation, access control bypass, integer overflow, frontrunning, and others—we provide the detection method, a Solidity example, the fix, and the tool that catches it. This is a practitioner’s manual: every technique described here is used in production on every Lux Network smart contract deployment.

Contents

1	The Audit Process	3
1.1	Why Audit	3
1.2	Five-Phase Methodology	3
1.3	Severity Classification	3
2	Phase 1: Static Analysis with Slither	3
2.1	What Slither Does	3
2.2	Running Slither	3
2.3	Key Detectors	4
2.4	False Positives	4
3	Phase 2: Pattern Detection with Aderyn	4
4	Phase 3: Fuzz Testing with Foundry	5
4.1	What Fuzzing Catches	5
4.2	Writing Fuzz Tests	5
4.3	Stateful Fuzzing (Invariant Testing)	5
4.4	Fuzzing Configuration	6
5	Phase 4: Symbolic Execution with Halmos	7
6	Phase 5: Manual Review	7
6.1	Manual Review Checklist	7
7	Vulnerability Taxonomy	7
7.1	Reentrancy	7
7.2	Flash Loan Attacks	8
7.3	Oracle Manipulation	8
7.4	Access Control	8
7.5	Integer Overflow	9
7.6	Frontrunning	9
8	Test Coverage	10
9	CI Integration	10
10	Conclusion	10

1 The Audit Process

1.1 Why Audit

Smart contracts are immutable programs that custody funds. A bug in a smart contract cannot be patched after deployment (without proxy patterns, which introduce their own risks). The asymmetry is extreme: the attacker needs to find one bug; the defender must eliminate all bugs.

1.2 Five-Phase Methodology

Our audit process has five sequential phases. Each phase catches different bug classes. Skipping any phase leaves a gap.

Phase	Method	Tool	Catches
1	Static analysis	Slither	Known patterns, code quality
2	Pattern detection	Aderyn	Domain-specific anti-patterns
3	Fuzz testing	Foundry	Implementation bugs, edge cases
4	Symbolic execution	Halmos	Logic bugs, invariant violations
5	Manual review	Human	Business logic, economic attacks

Table 1: The five-phase audit methodology.

1.3 Severity Classification

Severity	Count	Criteria
Critical	3	Direct fund loss, exploitable without special conditions
High	18	Indirect fund loss, requires specific conditions
Medium	37	Protocol malfunction, no direct fund loss
Low	41	Code quality, gas optimization, best practices
Informational	23	Style, documentation, non-functional
Total	122	

Table 2: Findings by severity across 50 contract domains.

2 Phase 1: Static Analysis with Slither

2.1 What Slither Does

Slither is a static analysis framework for Solidity. It parses Solidity source code into an intermediate representation (SlithIR) and runs detectors—pattern-matching rules that identify known vulnerability classes.

2.2 Running Slither

Listing 1: Running Slither on a Foundry project

```
1 # Install
2 pip install slither-analyzer
3
4 # Run all detectors
5 slither . --filter-paths "test/,script/,lib/"
6
7 # Run specific detector
```

```

8 slither . --detect reentrancy-eth
9
10 # Generate human-readable report
11 slither . --print human-summary
12
13 # Generate machine-readable JSON
14 slither . --json slither-report.json

```

2.3 Key Detectors

Detector	Vulnerability	Findings
reentrancy-eth	Reentrancy with ETH transfer	2
reentrancy-no-eth	Reentrancy without ETH transfer	5
uninitialized-state	Uninitialized storage variables	3
arbitrary-send-eth	Unchecked ETH transfers	4
controlled-delegatecall	Delegatecall with user input	1
suicidal	Unprotected selfdestruct	0
unchecked-transfer	Unchecked ERC-20 transfer	7
locked-ether	Contracts that receive but cannot send ETH	2

Table 3: Slither detector results across Lux Network contracts.

2.4 False Positives

Slither produces false positives. A finding is not a bug until confirmed by manual review. Our false positive rate across 50 domains: 34%. This means 66% of Slither findings are real issues. The remaining 34% are either intentional patterns (e.g., reentrancy guards that Slither does not recognize) or context-dependent non-issues.

3 Phase 2: Pattern Detection with Aderyn

Aderyn is a Rust-based Solidity analyzer that detects domain-specific anti-patterns that Slither misses. It is faster than Slither (Rust vs. Python) and focuses on patterns specific to DeFi, bridges, and governance.

Listing 2: Running Aderyn

```

1 # Install
2 cargo install aderyn
3
4 # Run analysis
5 aderyn .
6
7 # Output: report.md with findings by severity

```

Key patterns Aderyn detects that Slither misses:

- Centralization risks (single admin key controls protocol).
- Missing zero-address validation in constructors.
- Unsafe use of `block.timestamp` for time-sensitive logic.
- Missing event emissions for state changes.
- Inconsistent use of `SafeERC20`.

4 Phase 3: Fuzz Testing with Foundry

4.1 What Fuzzing Catches

Fuzzing generates random inputs and checks that invariants hold. It catches:

- Arithmetic overflow/underflow (despite Solidity 0.8 checks).
- Edge cases: zero amounts, maximum uint256, empty arrays.
- State-dependent bugs: invariant violations after specific transaction sequences.

4.2 Writing Fuzz Tests

Listing 3: Foundry fuzz test example

```
1 function testFuzz_depositWithdraw(  
2     uint256 amount,  
3     address user  
4 ) public {  
5     // Bound inputs to valid ranges  
6     amount = bound(amount, 1, 1e24);  
7     vm.assume(user != address(0));  
8     vm.assume(user != address(vault));  
9  
10    // Setup  
11    deal(address(token), user, amount);  
12    vm.startPrank(user);  
13    token.approve(address(vault), amount);  
14  
15    // Record state  
16    uint256 totalBefore = vault.totalAssets();  
17  
18    // Deposit  
19    uint256 shares = vault.deposit(amount, user);  
20    assertGt(shares, 0, "shares_must_be_nonzero");  
21  
22    // Withdraw  
23    uint256 withdrawn = vault.redeem(shares, user, user);  
24    vm.stopPrank();  
25  
26    // Invariant: withdrawn <= deposited (rounding loss ok)  
27    assertLt(withdrawn, amount, "cannot_withdraw_more_than_deposited");  
28    // Invariant: vault total assets are consistent  
29    assertEq(vault.totalAssets(), totalBefore, "total_assets_unchanged");  
30 }
```

4.3 Stateful Fuzzing (Invariant Testing)

Foundry's invariant testing mode generates random sequences of function calls and checks invariants after each call:

Listing 4: Stateful invariant test

```
1 // Handler contract: defines valid actions  
2 contract PoolHandler is Test {  
3     Pool public pool;  
4  
5     function swap(uint256 amount, bool direction) external {  
6         amount = bound(amount, 1, pool.reserve0() / 2);
```

```

7         if (direction) {
8             pool.swap(amount, 0, address(this));
9         } else {
10            pool.swap(0, amount, address(this));
11        }
12    }
13
14    function addLiquidity(uint256 amount0, uint256 amount1) external {
15        amount0 = bound(amount0, 1, 1e24);
16        amount1 = bound(amount1, 1, 1e24);
17        pool.addLiquidity(amount0, amount1, address(this));
18    }
19 }
20
21 // Invariant contract
22 contract PoolInvariantTest is Test {
23     PoolHandler handler;
24     Pool pool;
25
26     function setUp() public { /* ... */ }
27
28     // This runs after every random action sequence
29     function invariant_constantProduct() public {
30         assertGe(
31             pool.reserve0() * pool.reserve1(),
32             initialK,
33             "k_must_never_decrease"
34         );
35     }
36
37     function invariant_noFreeTokens() public {
38         assertEq(
39             token0.balanceOf(address(pool)),
40             pool.reserve0(),
41             "reserve0_matches_actual_balance"
42         );
43     }
44 }

```

4.4 Fuzzing Configuration

Listing 5: foundry.toml fuzz configuration

```

1 [fuzz]
2 runs = 10000           # Number of fuzz runs per test
3 max_test_rejects = 500 # Max invalid inputs before failing
4 seed = 0               # 0 = random seed each run
5 dictionary_weight = 40 # Weight for dictionary-based inputs
6
7 [invariant]
8 runs = 256             # Number of invariant test runs
9 depth = 128            # Max actions per run
10 fail_on_revert = false # Don't fail on expected reverts

```

5 Phase 4: Symbolic Execution with Halmos

Symbolic execution (covered in depth in our companion paper) proves properties for *all* inputs. In the audit context, we write Halmos tests for the most critical invariants:

- Token supply conservation.
- Bridge balance invariant (locked = minted on destination).
- Access control completeness (all admin functions revert for non-admins).
- Vault share/asset ratio monotonicity.

6 Phase 5: Manual Review

Automated tools catch *known* vulnerability patterns. Manual review catches:

- **Business logic errors:** The code does what it says, but what it says is wrong. Example: a fee calculation that charges 0.3% on the output instead of the input, giving arbitrageurs a profitable loop.
- **Economic attacks:** Flash loan attacks, oracle manipulation, sandwich attacks. These exploit *interactions between contracts* that automated tools analyze in isolation.
- **Governance attacks:** Vote buying, proposal frontrunning, timelock bypass.
- **Upgrade risks:** Proxy storage collisions, initialization order dependencies.

6.1 Manual Review Checklist

1. Read every line of code. Not skim—read.
2. For each external call: can the callee re-enter? What state is inconsistent at the call point?
3. For each `msg.sender` check: can it be bypassed via delegatecall or proxy?
4. For each arithmetic operation: what happens at the extremes (0, 1, `type(uint256).max`)?
5. For each oracle read: what if the oracle returns stale data? Zero? Negative?
6. For each token interaction: does it handle fee-on-transfer tokens? Rebasing tokens?
7. For each access control: is it consistent across all functions? Can admin keys be rotated?
8. For each storage write: is the order correct? Can it be frontrun?

7 Vulnerability Taxonomy

7.1 Reentrancy

Definition 1 (Reentrancy). *A reentrancy vulnerability exists when a contract makes an external call before updating its state, allowing the callee to call back into the contract and observe or modify inconsistent state.*

Listing 6: Reentrancy vulnerability and fix

```
1 // VULNERABLE
2 function withdraw(uint256 amount) external {
3     require(balances[msg.sender] >= amount);
4     (bool ok,) = msg.sender.call{value: amount}("");
5     require(ok);
6     balances[msg.sender] -= amount; // Too late!
7 }
8
9 // FIXED: checks-effects-interactions pattern
10 function withdraw(uint256 amount) external {
11     require(balances[msg.sender] >= amount);
12     balances[msg.sender] -= amount; // State update first
13     (bool ok,) = msg.sender.call{value: amount}("");
```

```

14     require(ok);
15 }

```

Detection: Slither reentrancy-eth, manual review.

7.2 Flash Loan Attacks

Flash loans allow borrowing unlimited capital for the duration of a single transaction. This enables attacks on any protocol that relies on spot prices or instantaneous balances.

Listing 7: Flash loan attack on a naive oracle

```

1 // VULNERABLE: uses spot price
2 function getPrice() public view returns (uint256) {
3     return reserve0 * 1e18 / reserve1; // Manipulable!
4 }
5
6 // FIXED: uses TWAP (time-weighted average price)
7 function getPrice() public view returns (uint256) {
8     (uint256 price, uint256 lastUpdate) = oracle.consult(
9         address(token), 1e18, 1800 // 30-minute TWAP
10    );
11    require(block.timestamp - lastUpdate < 3600, "stale");
12    return price;
13 }

```

Detection: manual review (automated tools cannot reason about cross-contract economic interactions).

7.3 Oracle Manipulation

Listing 8: Oracle manipulation defense

```

1 // VULNERABLE: single oracle, no staleness check
2 function liquidate(address user) external {
3     uint256 price = oracle.latestAnswer();
4     require(debt(user) * price > collateral(user) * 1e18);
5     // liquidate...
6 }
7
8 // FIXED: multiple oracles, staleness check, deviation check
9 function liquidate(address user) external {
10    (uint256 price1, uint256 ts1) = oracle1.latestRoundData();
11    (uint256 price2, uint256 ts2) = oracle2.latestRoundData();
12    require(block.timestamp - ts1 < 3600, "oracle1_␣stale");
13    require(block.timestamp - ts2 < 3600, "oracle2_␣stale");
14    require(
15        diff(price1, price2) * 100 / price1 < 5,
16        "oracle_␣deviation"
17    );
18    uint256 price = (price1 + price2) / 2;
19    require(debt(user) * price > collateral(user) * 1e18);
20    // liquidate...
21 }

```

7.4 Access Control

Listing 9: Access control vulnerability and fix

```
1 // VULNERABLE: missing access control
2 function setFee(uint256 newFee) external {
3     fee = newFee; // Anyone can call this!
4 }
5
6 // FIXED: proper access control
7 function setFee(uint256 newFee) external onlyOwner {
8     require(newFee <= MAX_FEE, "fee too high");
9     emit FeeUpdated(fee, newFee);
10    fee = newFee;
11 }
```

Detection: Slither missing-zero-check, Aderyn centralization detector.

7.5 Integer Overflow

Solidity 0.8+ has built-in overflow checks, but unchecked blocks bypass them:

Listing 10: Overflow in unchecked block

```
1 // VULNERABLE: overflow in unchecked block
2 function accumulateFees(uint256 newFees) internal {
3     unchecked {
4         totalFees += newFees; // Can wrap around!
5     }
6 }
7
8 // FIXED: only use unchecked where overflow is proven impossible
9 function accumulateFees(uint256 newFees) internal {
10    totalFees += newFees; // Solidity 0.8 overflow check
11 }
```

7.6 Frontrunning

Listing 11: Frontrunning defense with commit-reveal

```
1 // VULNERABLE: swap with no slippage protection
2 function swap(uint256 amountIn) external returns (uint256) {
3     return _swap(amountIn); // Sandwich-able
4 }
5
6 // FIXED: minimum output amount
7 function swap(
8     uint256 amountIn,
9     uint256 minAmountOut,
10    uint256 deadline
11 ) external returns (uint256 amountOut) {
12    require(block.timestamp <= deadline, "expired");
13    amountOut = _swap(amountIn);
14    require(amountOut >= minAmountOut, "slippage");
15 }
```

8 Test Coverage

Domain	Contracts	Tests	Branch %	Findings
AMM/DEX	8	147	96%	14
Bridge (Teleport)	6	98	94%	18
Yield vaults	5	72	91%	11
Staking	4	63	93%	8
Governance	7	85	89%	15
Oracle	3	41	95%	7
Token (ERC-20/721)	6	94	97%	12
Lending	4	67	90%	13
Other (33 domains)	45	165	88%	24
Total	88	832	92%	122

Table 4: Test coverage and findings across 50 contract domains.

9 CI Integration

Every pull request to a Lux Network smart contract repository runs the full audit pipeline:

Listing 12: CI audit pipeline

```
1 audit:
2   steps:
3     - name: Build
4       run: forge build --sizes
5     - name: Unit tests
6       run: forge test -vvv
7     - name: Coverage
8       run: forge coverage --report lcov
9     - name: Slither
10      run: slither . --filter-paths "test/,script/,lib/"
11          --fail-on medium
12     - name: Aderyn
13       run: aderyn .
14     - name: Halmos
15       run: halmos --function check_ --loop 5
16     - name: Gas snapshot
17       run: forge snapshot --check
```

The pipeline fails if:

- Any test fails.
- Branch coverage drops below 85%.
- Slither reports a medium-or-higher finding.
- Any Halmos symbolic test fails.
- Gas usage increases by more than 10% from the snapshot.

10 Conclusion

Smart contract auditing is not a single technique—it is a pipeline. Static analysis catches known patterns. Fuzzing catches implementation edge cases. Symbolic execution proves invariants. Manual review catches business logic errors. Each phase catches bugs that the others miss.

Our methodology, applied to 50 contract domains with 832 tests, found 122 issues including 3 critical vulnerabilities that would have resulted in direct fund loss. All findings were fixed

before deployment. The entire pipeline runs automatically on every pull request, ensuring that new code does not introduce regressions.

The most important lesson: no tool replaces reading the code. Tools find *known* bug patterns efficiently. Humans find *unknown* bug patterns—the ones that matter most.

References

- [1] Feist, J. et al. “Slither: A Static Analysis Framework For Smart Contracts.” WOOT, 2019.
- [2] Paradigm. “Foundry: A Blazing Fast, Portable and Modular Toolkit for Ethereum Application Development.” 2022.
- [3] Park, D. et al. “Halmos: Symbolic Testing for EVM Smart Contracts.” a16z Crypto, 2023.
- [4] Cyfrin. “Aderyn: Rust-Based Solidity AST Analyzer.” 2024.
- [5] NCC Group. “Decentralized Application Security Project (DASP) Top 10.” 2018.
- [6] SmartContractSecurity. “SWC Registry: Smart Contract Weakness Classification.” 2020.